Orientation-Aware Seam Carving for Image Retargeting

Shreyas Gupta
230991
EE604, Department of Electrical Engineering
Indian Institute of Technology Kanpur

Abstract

Image retargeting changes the height and width of a photograph so that it fits a new layout while the objects a viewer would notice, and the long straight structures in the scene, stay intact. Uniform scaling compresses the whole frame, and a center crop throws away everything outside a window. This project instead deletes or duplicates low-cost seams. The seam cost is the forward energy of Rubinstein, Shamir, and Avidan, which charges the new edges a deletion would create, plus two maps that use no learned weights: spectral-residual saliency, and a line map whose weight depends on the direction of the seam. Vertical seams are discouraged from cutting vertical structures, and horizontal seams from cutting horizontal ones. On eight images from the RetargetMe benchmark, the combined cost is compared with scaling, center-cropping, classic backward-energy seams, and ablations that turn saliency or lines off. Against forward energy alone, on eight images at a 25% width reduction, the saliency weight lowers the mean deleted spectral-residual fraction from 0.089 to 0.068, and the line weight lowers crossings of the same line map from 1.14 to 0.31 per 100 seam pixels. The full model matches the saliency-only score and sits a little above the lines-only score, at 0.37 rather than 0.31. Both scores reuse the maps inside the cost, so they show that the weights move the seams. They are not an independent measure of whether the picture looks better. At 75% width the two seam results look similar. A center crop already removes off-center content, and a 50% cut of a curved sculpture pinches it under both seam methods. A hybrid schedule uses the same cost but stops the seams at 75% of the original side and scales the rest. The 50% result is then the 75% seam image scaled down. It does not add the further pinching that comes from carving all the way to half width, and it still does not match uniform scaling, which never cuts the sculpture.

1. Introduction

Photographs are displayed at aspect ratios their cameras did not use: a phone story, a banner, a slide, a print. The operation that produces the new size is image retargeting. Its input is one color image, a target height, and a target width. Its output is an image of that exact size. The constraint in this project is that the computation stays in the spatial domain. There is no trained retargeting network and no learned saliency model. The implementation uses NumPy and OpenCV, and the dynamic program runs on images whose long side is at most 640 pixels so that a demo can recompute a result on request.

Two standard operators ignore content. Scaling moves every pixel by the same factor, so a face becomes thinner when the frame becomes narrower. Cropping keeps a window and discards the rest, so an off-center subject can disappear even though the remaining pixels look sharp. Seam carving [1] takes a third route. A vertical seam is an 8-connected path of one pixel per row. Deleting the cheapest such paths changes the width without a global resample. The original cost is a backward energy: the gradient magnitude of the pixels that are removed. That cost does not look at the tear those pixels leave behind, and a smooth subject can be cheaper to cut than a textured background.

Forward energy [3] fixes the tear by scoring the edges that appear after the deletion. It still has no notion of a person or of a window mullion. Saliency raises the cost of pixels that the spectral residual marks. Those pixels are often edges, not a detected person. A line map raises the cost of long straight segments. Both ideas exist in the literature as separate add-ons. The variant defended here puts them in one recurrence, and it does not treat every line the same way. A vertical seam is allowed to cross a horizon, because every vertical seam must, and it is penalized for crossing a column or a mullion. A horizontal seam swaps that preference. Section 3 defines the cost. Section 4 compares it with scaling, cropping, and classic seams, and turns each

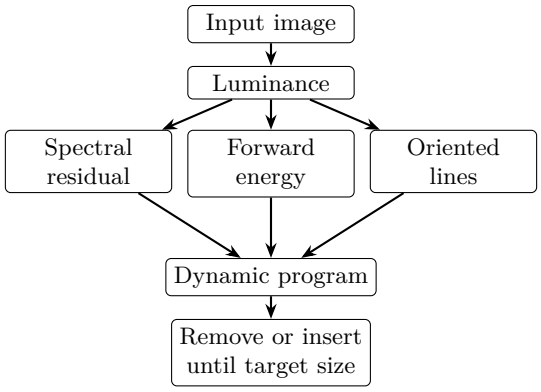


Figure 1. Overview. Forward energy is evaluated on the current luminance. Saliency and the line maps travel with the pixels as seams are removed.

extra term off in an ablation.

2. Related work

Avidan and Shamir [1] select seams by dynamic programming on an energy map, remove them to shrink, and duplicate them to enlarge. The energy they use is a function of the image gradient. Rubinstein, Shamir, and Avidan [3] replace that energy with a forward cost that depends on which neighbor the seam arrives from, because the pixels that become adjacent are not known from the gradient alone. The same authors later choose among scaling, cropping, and seam carving with a multi-operator search [4]. We do not search over operators. Scaling and cropping are baselines, and the seam operator is fixed.

Hou and Zhang [2] define a saliency map from the spectral residual of the Fourier amplitude. The residual is the log amplitude minus a local average of the log amplitude, combined with the original phase. The map needs no training images. RetargetMe [5] is the public benchmark used below. It compares several retargeting operators with a user study. We report automatic scores and side-by-side figures on a subset of its source images, and we do not repeat that user study.

3. Method

Figure 1 is the pipeline. Luminance is computed once per seam from the current color image. Saliency and the two line maps are computed once on the image at the start of a shrinking or enlarging stage. When a seam is deleted, the same pixel indices are deleted from those maps. Forward energy is not stored. It is recomputed from the current luminance inside the dynamic program, because it depends on the neighbors that still exist.

3.1. Saliency and line protection

Luminance is the Rec. 601 combination $Y = 0.299R + 0.587G + 0.114B$, scaled to $[0, 1]$. Spectral residual saliency follows Hou and Zhang. The log amplitude of the Fourier transform is subtracted from its 3×3 local average, the original phase is kept, and the inverse transform is squared, blurred with a Gaussian of standard deviation 2.5, and normalized to $[0, 1]$.

Line segments come from a blurred Canny edge map and the probabilistic Hough transform. The blur (a 5×5 Gaussian with $\sigma = 1.2$) and the Canny thresholds (80 and 160) are there so that brick texture does not fill the mask. A segment is kept only when it is at least 30 pixels long and at least 12% of the short side. At most 40 of the longest segments are drawn, with thickness 2. Let θ be the segment angle from the x-axis. For a vertical seam, which changes the width, the segment is drawn with weight $|\sin\theta|$. A vertical mullion then has weight 1, and a horizontal horizon has weight 0. For a horizontal seam the weight is $|\cos\theta|$. Overlapping draws keep the per-pixel maximum. The horizon still has to be crossed by every vertical seam. The weight stops that forced crossing from also penalizing columns and poles.

3.2. Dynamic program

Index rows by i and columns by j, and write I for the current luminance. The forward costs of arriving at (i, j) from the three pixels in the row above are

$$C_U(i,j) = |I(i,j{+}1) - I(i,j{-}1)|, \tag{1}$$
$$C_L(i,j) = C_U(i,j) + |I(i{-}1,j) - I(i,j{-}1)|, \tag{2}$$
$$C_R(i,j) = C_U(i,j) + |I(i{-}1,j) - I(i,j{+}1)|. \tag{3}$$

C_U is the new edge between the left and right neighbors that the deletion joins. C_L and C_R add the edge against the pixel the seam just left when the step is diagonal. Out-of-range neighbors are clamped to the border pixel. A step that would leave the image is given infinite cost. The direction-independent term is the saliency S and the line map L_d for the seam direction d,

$$E(i,j) = w_s S(i,j) + w_l L_d(i,j). \tag{4}$$

The cumulative cost and the chosen step are

$$M(i,j) = E(i,j) + \min\big\{M(i{-}1,j{-}1) + C_L,\; M(i{-}1,j) + C_U,\; M(i{-}1,j{+}1) + C_R\big\}. \tag{5}$$

The first row is $M(0, j) = E(0, j)$. Backtracking from the minimum of the last row yields one column index per row. Consecutive indices differ by at most one, so

the seam is 8-connected. The weights used for every reported number are $w_s = 0.5$ and $w_l = 0.8$. They were chosen by looking at Johanneskirche and obama, which are two of the eight evaluation images, and then frozen. There is no weight sweep. The demo sliders can still override them. Section 4 also reports the means with those two images removed, as a sensitivity check rather than a held-out tuning.

Classic seam carving uses the same backtracking with a different recurrence. Its pixel cost is the sum of absolute Sobel responses, and it has no forward term and no saliency or line term. Horizontal seams are vertical seams on the transposed image, with the matching line map transposed as well. While both the height and the width are still above the target, the algorithm computes one seam in each direction and deletes the cheaper one. Enlargement records the seams that would be removed, maps their indices back onto the image before insertion, and writes an averaged copy of each seam pixel next to its right neighbor. One recording pass inserts at most $W-2$ seams, which can almost double the width. The $1.5\times$ enlargement in the experiments fits in that single pass. A second pass runs only when the target is wider than $2W - 2$.

3.3. Hybrid schedule

The full model above carves until the image is the requested size. On a deep shrink that is where curved, busy scenes tear: each new seam is still the cheapest path, and after the easy background is gone the path goes through the object. The hybrid operator keeps Equation (5) and the same weights, and changes only when scaling takes over. If a target side is at least 75% of the original, hybrid is the full model and does not scale. If the target is smaller, seams stop at round($0.75 \times$ original) and cv2.resize finishes that side. A request for 50% width therefore deletes columns by seams until the width is 75% of the original, then resizes that image to 50%. Enlargement is unchanged. This is a fixed schedule, not a search over operators [4]. An edge-density term was tried as a way to protect curves and then left out of the cost. A vertical seam crosses a curved outline in a few rows and then travels through the smooth interior, so raising the outline moves the seam into that interior. On Sanfrancisco those paths go through the painted sky on the heart. The Hough line map is also unchanged. OpenCV's line-segment detector is present in this installation and was not substituted, so every line score in Section 4 still refers to one detector.

4. Experiments

4.1. Data and implementation

The images are eight source photographs from RetargetMe [5]: obama and child (people), brick_house and Johanneskirche (buildings), eagle and butterfly (compact objects), waterfall (a landscape), and Sanfrancisco (a busy outdoor scene). The download script keeps these eight files from the public source-image archive and does not redistribute the other 72. Before any operator runs, the long side is limited to 640 pixels with area interpolation. The code is Python 3 with NumPy and OpenCV. No GPU is used. The row update of the dynamic program is vectorized across the width.

Five operators are compared. Scale is cv2.resize to the exact target, with area interpolation when both sides shrink. Crop cuts the centered window of the target aspect ratio and then resizes it to the exact target. Classic is backward-energy seam carving. Ours is Equation (5) with $w_s = 0.5$ and $w_l = 0.8$. Hybrid is that same cost, with seams stopped at 75% of the original side and a uniform scale after that. At a 75% target the two are the same operator. Hybrid is reported on the 50% width cut. The ablation uses the same recurrence with (w_s, w_l) set to $(0, 0)$ (forward energy alone), $(0.5, 0)$ (saliency only), and $(0, 0.8)$ (lines only).

The width is reduced to 75% and to 50% of the working width, with the height held fixed. The 75% setting is a 25% reduction and is the one used for the ablation and the runtime. waterfall is also reduced to 75% height, and butterfly is enlarged to 150% width. Every reported output was checked to have the requested height and width.

Two seam scores are reported, and both are computed from the maps that the full model puts in its cost. The saliency fraction is the sum of the spectral-residual map on pixels that are actually deleted, divided by the sum of that map. Insertion does not delete pixels, so the fraction is not used for the enlargement. The line score is the number of seam pixels that land on a positive value of that same line map, per 100 seam pixels. Classic seams are scored with the map even though their recurrence does not use it. A lower number therefore shows that the weight changed which pixels were deleted. It does not, by itself, show that a viewer would prefer the result, and it does not measure tears in regions the maps do not mark. Scale and crop do not choose seams, so they have no entry on these two scores. Runtime is one wall-clock measurement of the whole call.



Figure 2. Maps on Johanneskirche, the line example. Left to right: the working image, spectral-residual saliency, the line map used by vertical seams (bright yellow marks near-vertical segments), and the next seam chosen by the full model.

Table 1. Eight images at 75% width. Entries are mean (sample standard deviation). Saliency is the fraction of spectral-residual mass deleted. Lines are pixels of the same line map per 100 seam pixels. Seconds are one run per image.

Method	Saliency	Lines	Seconds
Scale	–	–	0.00
Crop	–	–	0.00
Classic	0.089 (0.045)	1.30 (1.25)	11.0 (1.5)
Forward only	0.089 (0.039)	1.14 (1.18)	13.7 (1.4)
Forward + saliency	0.068 (0.035)	0.99 (1.09)	12.9 (3.0)
Forward + lines	0.089 (0.040)	0.31 (0.22)	13.4 (3.3)
Full model	0.068 (0.036)	0.37 (0.31)	14.4 (1.7)

Table 2. The same eight images at 50% width, mean (sample standard deviation).

Method	Saliency	Lines	Seconds
Scale	–	–	0.00
Crop	–	–	0.00
Classic	0.222 (0.097)	1.61 (1.34)	21.5 (3.9)
Full model	0.186 (0.090)	0.69 (0.56)	26.0 (3.3)

4.2. Results

Table 1 gives the mean and sample standard deviation at 75% width. Table 2 is the 50% width cut. Table 3 is the per-image breakdown at 75% width. Figure 3 shows four of the eight images at 75% width: the church (lines), the portrait (saliency), the street sculpture (a busy scene), and the waterfall. Figure 4 repeats that set at 50% width. Figure 5 isolates the terms on the church. Figure 6 is the 50% cut of Sanfrancisco. Figure 8 and Figure 9 are the height reduction and the enlargement.

At a 25% width reduction the two extra terms move different columns of Table 1, and the standard deviations are large. Forward energy alone and classic backward energy both have mean saliency fraction 0.089. Adding saliency lowers that mean to 0.068. Adding the line map without saliency leaves the saliency mean at 0.089 and lowers the line score from 1.14 to 0.31. The full model matches the saliency-only mean, 0.068, so on this score the line weight does not undo the saliency weight. On the line score the full model is 0.37, which is still far below forward energy and a little worse than lines alone. That is most of the line-term reduction, not all of it. Table 3 shows why several means print as the same three-decimal number. The per-image fractions differ. The agreement is rounding of the average. The full model is below classic on saliency fraction for every image. On line crossings it is lower on seven images. On child both scores are at most 0.01, because that photograph barely fires the line detector. The absolute line counts are small because the map is sparse: 1.14 versus 0.31 crossings per 100 pixels is a few marked pixels on a seam of a few hundred rows, and the classic standard deviation, 1.25, is as large as the classic mean. At 50% width the means move the same way, 0.186 against 0.222 for saliency and 0.69 against 1.61 for lines, and the full model is lower on both scores for every image. The spread remains large.

The weights were chosen on Johanneskirche and obama. Dropping those two and averaging the other six, at 75% width, leaves the same separation: classic 0.096 and 1.45, forward 0.095 and 1.19, saliency only 0.074 and 1.01, lines only 0.095 and 0.28, full model 0.075 and 0.33, again as saliency fraction and line crossings. This only checks that the direction is not confined to the two inspected images. It is not a retune on held-out data.

Figures 3 and 4 are qualitative. At 75% width, classic seams and the full model look close to each other, while scaling compresses the frame and the center crop cuts the heart in Sanfrancisco and clips the child in obama. At 50% width the sheets suggest less bending in the church window and in the child's outline under the full model. Those sheets are small, and this paper does not treat that impression as a measurement. Sanfrancisco is a failure under both seam methods: the heart is curved, the Hough map does not guard it, and both pinch it. The crop cuts it off, and scaling keeps it by squeezing the frame. Figure 7 is the same 50% width with the hybrid schedule added. The hybrid column is the 75% seam result scaled down to half width. It does not receive the extra seams that, under the full model,

Table 3. Per-image scores at 75% width, written as saliency fraction / line crossings per 100 seam pixels. Means that agree to three decimals in Table 1 are not the same seams. Images marked † are the two used to choose w_s and w_l.

Image	Classic	Forward	+Saliency	+Lines	Full
brick_house	0.153/3.23	0.142/1.82	0.120/1.62	0.142/0.33	0.124/0.46
butterfly	0.043/0.16	0.055/0.36	0.037/0.22	0.052/0.04	0.036/0.07
child	0.023/0.00	0.033/0.13	0.020/0.03	0.032/0.07	0.020/0.01
eagle	0.108/1.62	0.094/0.47	0.075/0.28	0.090/0.13	0.074/0.13
Johanneskirche†	0.070/0.94	0.071/0.97	0.055/0.90	0.071/0.35	0.055/0.38
obama†	0.069/0.74	0.066/1.02	0.042/0.97	0.067/0.47	0.042/0.57
Sanfrancisco	0.137/3.09	0.134/3.76	0.108/3.37	0.139/0.71	0.109/0.95
waterfall	0.110/0.60	0.114/0.59	0.086/0.52	0.114/0.38	0.087/0.37

continue from 75% to 50% and cut the heart further. It is also not the scaled original: the first 25% of seams have already narrowed the sculpture relative to uniform scaling. The saliency and line scores are not quoted for hybrid. Those scores count only the seams, and hybrid deletes half as many seams before the scale, so a lower fraction would not mean a better picture. The waterfall height cut and the butterfly enlargement are single images. On the height cut the horizontal line score is 0 for both seam methods, so that map did nothing there; the saliency fractions are 0.009 and 0.016, which is one pair of numbers, not a mean. The enlargement reaches 150% width. Scaling stretches the butterfly. Seam insertion spends the extra width on the background by writing averaged copies of the recorded seams.

A 25% width cut takes about 11 s for classic seams and about 14 s for the forward variants. Scaling and cropping finish in well under 0.01 s. The maps are computed once per call, so they do not explain the differences among the forward rows in Table 1. Those differences sit inside the run-to-run spread of a single timing per image. The steadier gap is between classic seams and any forward recurrence, which is recomputed on every seam.

The full eight-image contact sheets are Figures 10 and 11 in the appendix. Across the 75% and 50% trials, every output size matched the requested size.

4.3. What the demo shows

The Gradio application loads an image, caps the long side at 640 pixels, and takes the target size as a percentage of that working image. The user picks scale, crop, classic, the full model, or the hybrid schedule, and moves w_s and w_l. Optional panels show the saliency map, the vertical-seam line map, and the next vertical seam. Three examples are wired to the roles above: Johanneskirche for lines, obama for a person, and Sanfrancisco for the busy failure. A useful order in a presentation is to raise w_l on the church, compare classic and the full model on the portrait, and then set the street scene to 50% width and say plainly what every method loses.

4.4. Limitations

The saliency and line scores reuse the maps inside the cost. A lower score shows that the corresponding weight changed the seam. It does not show that the deleted pixels were less important to a viewer, and pushing seams off those maps can tear texture that neither map marks. No second saliency model, no independent line detector, and no summed forward-energy of the chosen seams is reported, so that tradeoff is not measured. Spectral residual saliency responds strongly to edges. It is not a subject detector. The line map only contains segments the Hough transform returns, so short and curved structures, including the heart sculpture, are unprotected except where the residual happens to be high. A strong-edge penalty does not fix that curve: the seam pays it at the crossing and then follows the cheap interior. The hybrid schedule stops further seam cuts after 75%, which is not the same as protecting the object during the first 25%. The weights were chosen on two of the eight test images, and they were not swept. The line-score standard deviation is about as large as the classic mean, and $n = 8$ is a small slice of RetargetMe. The comparison does not include the published RetargetMe operators, such as multi-operator search, shift-map, or warping. Hybrid is a fixed seam-then-scale schedule, not that search. The height cut and the enlargement are single images. Runtimes are one measurement each. There is no user study.

5. Conclusion

The operator is forward-energy seam carving plus a spectral-residual term and an orientation-weighted line term. On eight RetargetMe images, turning the terms off one at a time moves two different scores, and both scores are built from the maps in the cost. The saliency weight lowers deleted residual mass from 0.089 to 0.068. The line weight lowers line-map crossings from 1.14 to 0.31 per 100 seam pixels. The full model keeps the saliency-score reduction and most, not all, of the line-score reduction: its line score is 0.37 rather than 0.31.

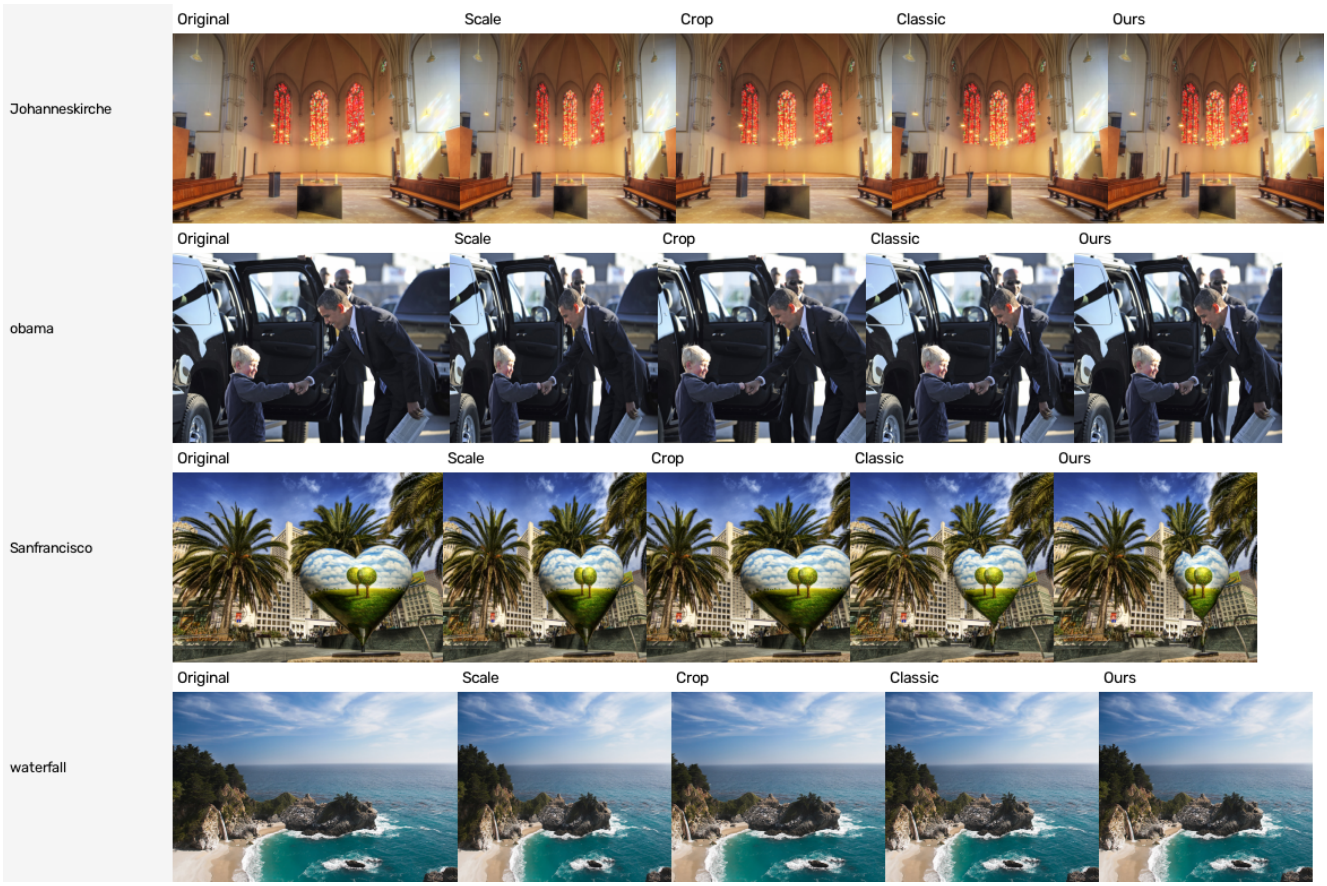


Figure 3. Width reduced to 75%. Columns are the original, uniform scale, center crop, classic seams, and the full model.

The same direction remains if the two images used to pick the weights are left out of the average. That is evidence that the weights do what they were built to do. It is not evidence that the results look better, that subjects are preserved, or that the method beats other published retargeting operators. Scaling and cropping are the simple baselines. A large reduction of a busy curved scene still damages the content. Stopping the seams at 75% and scaling the rest removes only the extra damage that comes from carving on to half width. It does not put the curved sculpture back to the shape that uniform scaling keeps.

References

[1] Shai Avidan and Ariel Shamir. Seam carving for content-aware image resizing. In ACM SIGGRAPH 2007 Papers, pages 10:1–10:9, 2007. 1, 2

[2] Xiaodi Hou and Liqing Zhang. Saliency detection: A spectral residual approach. In IEEE Conference on Computer Vision and Pattern Recognition, pages 1–8, 2007. 2

[3] Michael Rubinstein, Ariel Shamir, and Shai Avidan. Improved seam carving for video retargeting. ACM Transactions on Graphics, 27(3):16:1–16:9, 2008. 1, 2

[4] Michael Rubinstein, Ariel Shamir, and Shai Avidan. Multi-operator media retargeting. ACM Transactions on Graphics, 28(3):23:1–23:11, 2009. 2, 3

[5] Michael Rubinstein, Diego Gutierrez, Olga Sorkine, and Ariel Shamir. A comparative study of image retargeting. ACM Transactions on Graphics, 29(6):160:1–160:11, 2010. 2, 3

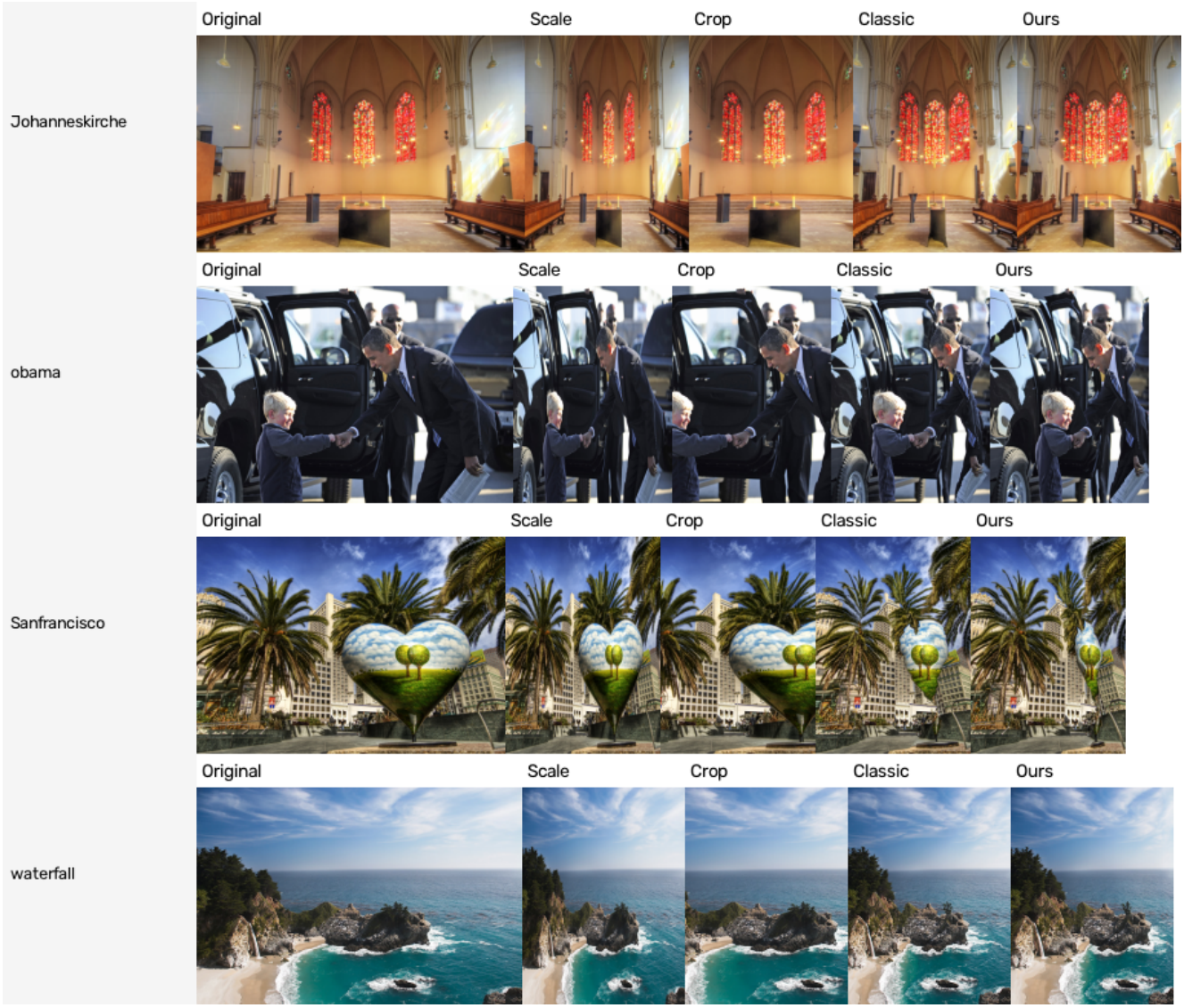


Figure 4. Width reduced to 50% on the same four images. Any impression that one seam method bends a window or an outline less than the other is qualitative. The measured claims are the scores in Tables 1–3.



Figure 5. Ablation on Johanneskirche at 75% width. Forward energy alone, forward energy plus saliency, forward energy plus lines, the full model, and classic backward energy.

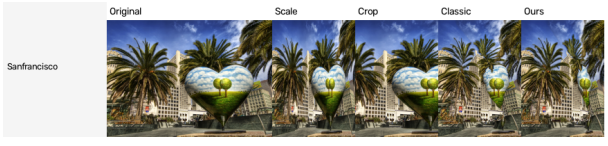


Figure 6. Failure case. Sanfrancisco at 50% width. The crop cuts the heart, scaling squeezes it, and both seam methods pinch the curved outline. The Hough map does not protect that curve.

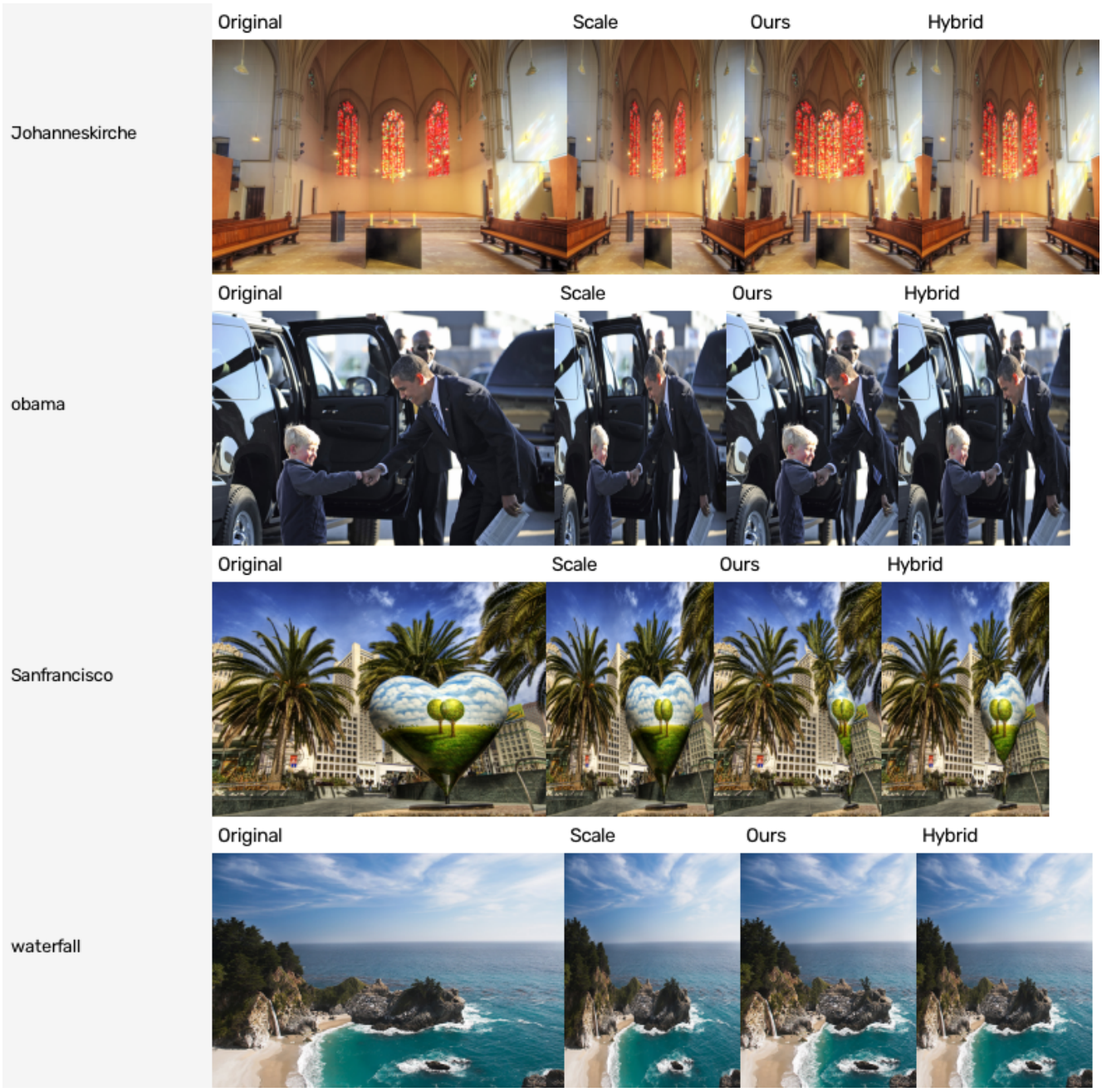


Figure 7. Width reduced to 50%. Columns are the original, uniform scale, the full model carved all the way, and hybrid (seams until 75% of the original width, then scale). Hybrid avoids only the damage that the extra seams would add past 75%.



Figure 8. Height reduced to 75% on waterfall. Horizontal seams use the line map weighted by $|\cos\theta|$.



Figure 9. Width enlarged to 150% on butterfly. Seams are recorded by removal and inserted as averaged duplicates.

A. Individual contributions

A.1. Shreyas Gupta (230991)

I implemented the luminance, spectral-residual saliency, and orientation-aware line maps; the forward-energy and classic backward-energy dynamic programs, including seam removal and insertion; the scaling and center-crop baselines; the hybrid schedule that stops seams at 75% of a side; the RetargetMe experiments and figures; the Gradio demo; and this write-up.

B. Links

The source images are the RetargetMe archive of Rubinstein et al.: http://people.csail.mit.edu/mrub/retargetme/ and the file linked as "source images only" from http://people.csail.mit.edu/mrub/retargetme/download.html. The libraries are OpenCV (https://github.com/opencv/opencv), NumPy (https://github.com/numpy/numpy), and Gradio (https://github.com/gradio-app/gradio). The complete source is printed in the next section and

included with this submission. No public repository link is attached.

C. All eight images

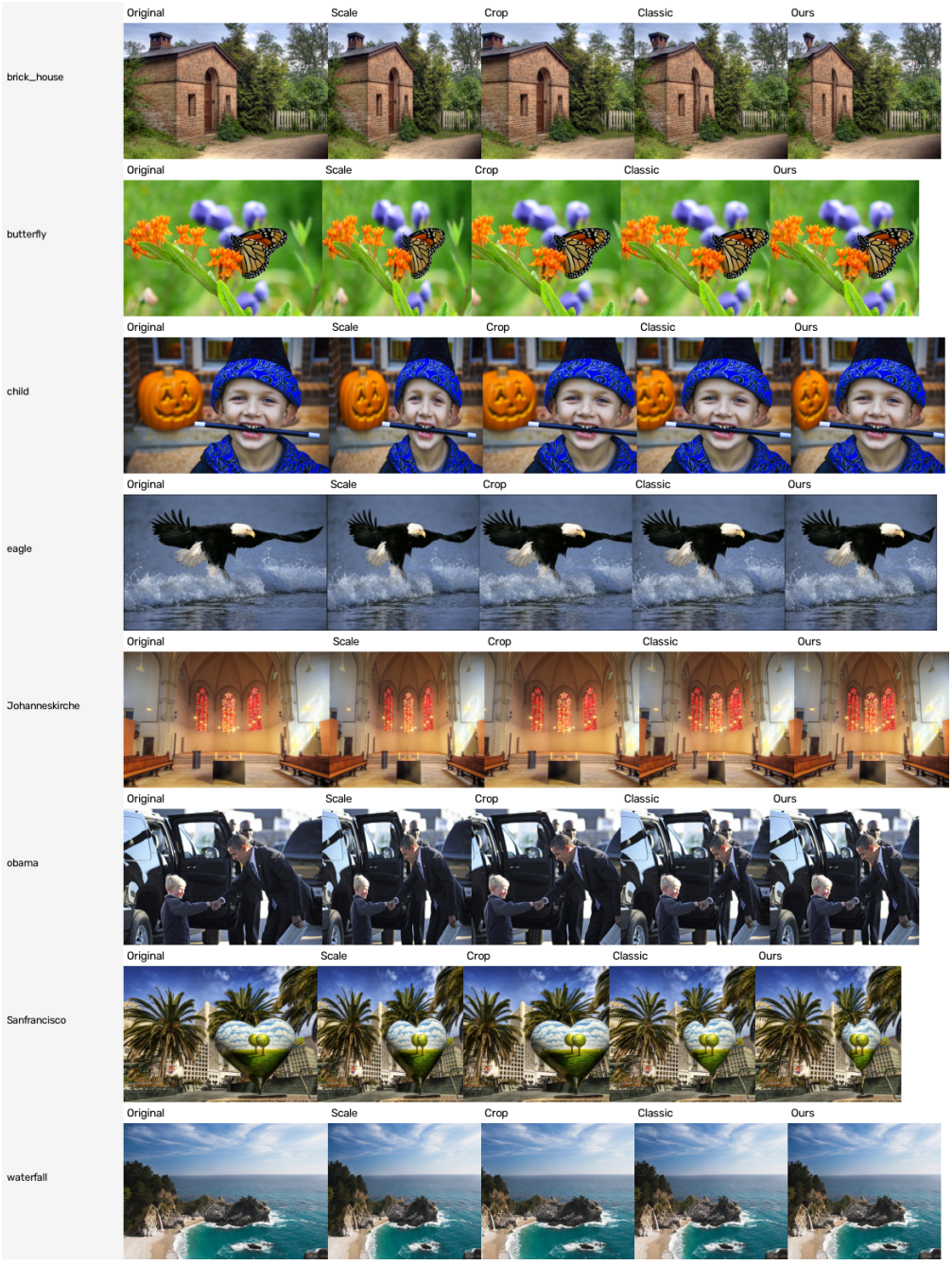


Figure 10. All eight images at 75% width. Columns are the original, scale, crop, classic seams, and the full model.

D. Source code

The listings below are the program submitted with this report. They match the source included with this submission.

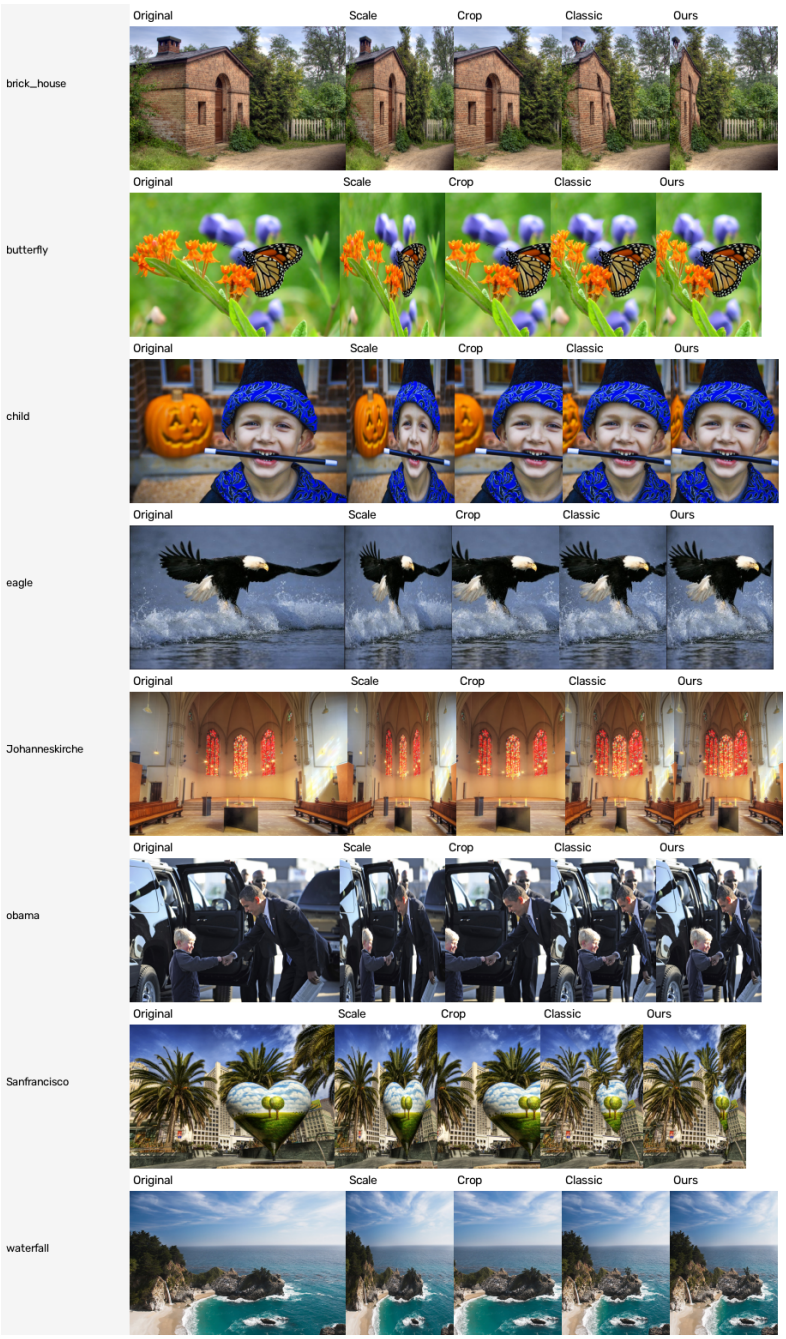


Figure 11. All eight images at 50% width.

```
"""Spatial-domain image retargeting."""

from .retarget import retarget

__all__ = ["retarget"]
```

Listing 1. src/__init__.py

```
"""Energy maps for content-aware image retargeting.

```

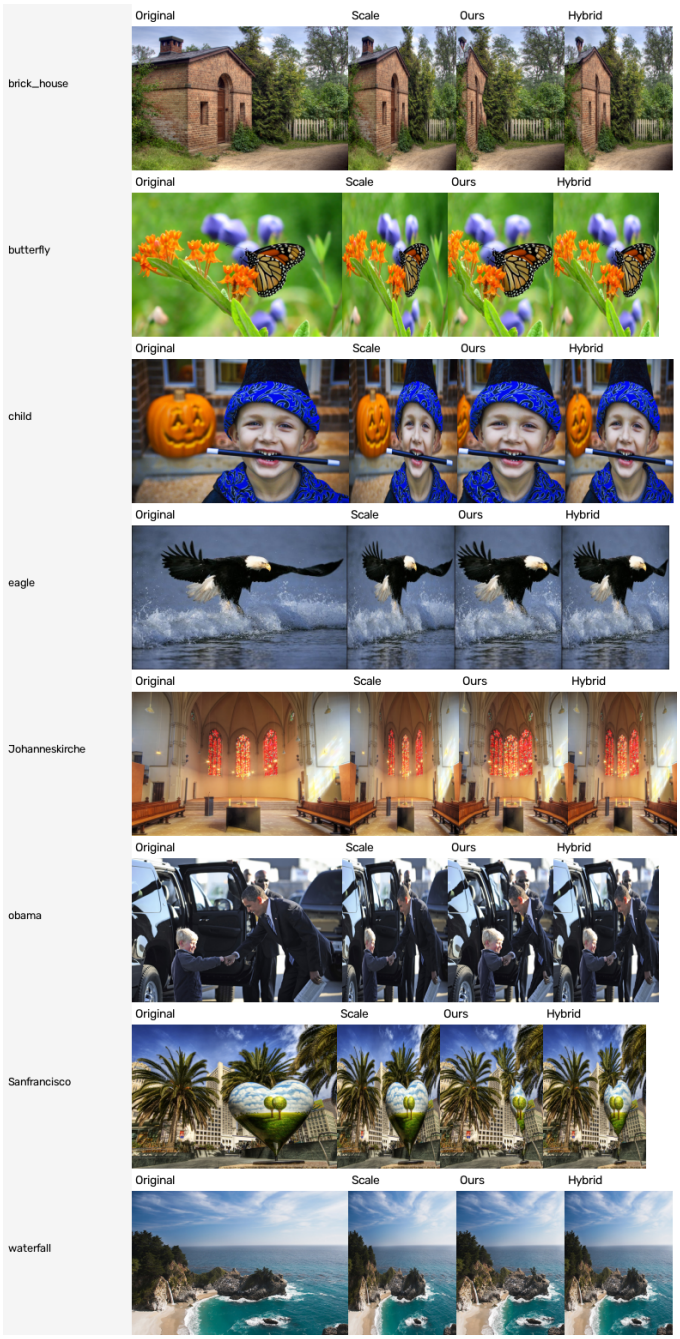


Figure 12. All eight images at 50% width. Columns are the original, uniform scale, the full model, and hybrid.

```
The dynamic program uses three maps:

* luminance, for the forward-energy recurrence
* spectral-residual saliency (Hou and Zhang, CVPR 2007)
* an orientation-aware line-protection map

‘‘curve_protection‘‘ builds a strong-edge map that includes curves.
It is not added to the cost: a seam crosses the outline and then runs
through the smooth interior. Saliency and lines are computed once.
As seams are deleted, the same pixel indices are deleted from those maps.
Forward energy is recomputed from the current luminance.
"""
```

```

from ___future___ import annotations

import cv2
import numpy as np


def luminance(image: np.ndarray) -> np.ndarray:
    """Rec. 601 luminance in ``[0, 1]``. ``image`` is RGB uint8 or float."""
    rgb = image.astype(np.float32)
    if rgb.max() > 1.0:
        rgb = rgb / 255.0
    if rgb.ndim == 2:
        return rgb
    r, g, b = rgb[:, :, 0], rgb[:, :, 1], rgb[:, :, 2]
    return 0.299 * r + 0.587 * g + 0.114 * b


def spectral_residual_saliency(lum: np.ndarray) -> np.ndarray:
    """Spectral residual saliency, normalized to ``[0, 1]``.

    The residual is the log amplitude of the Fourier transform minus a
    local average of that log amplitude. The original phase is kept, and
    the inverse transform is squared and blurred. No learned weights.
    """
    gray = lum.astype(np.float32)
    spectrum = np.fft.fft2(gray)
    log_amplitude = np.log(np.abs(spectrum) + 1e-8)
    phase = np.angle(spectrum)
    # 3x3 local average, as in Hou and Zhang.
    average = cv2.blur(log_amplitude.astype(np.float32), (3, 3))
    residual = log_amplitude - average
    reconstructed = np.fft.ifft2(np.exp(residual + 1j * phase))
    saliency = np.abs(reconstructed).astype(np.float32) ** 2
    saliency = cv2.GaussianBlur(saliency, (9, 9), 2.5)
    return _normalize(saliency)


def gradient_magnitude(lum: np.ndarray) -> np.ndarray:
    """Backward-energy map: sum of absolute Sobel responses."""
    gx = cv2.Sobel(lum.astype(np.float32), cv2.CV_32F, 1, 0, ksize=3)
    gy = cv2.Sobel(lum.astype(np.float32), cv2.CV_32F, 0, 1, ksize=3)
    return np.abs(gx) + np.abs(gy)


def curve_protection(lum: np.ndarray) -> np.ndarray:
    """Strong-boundary map in ``[0, 1]``, including curves.

    Not added to the seam cost. A vertical seam crosses a curved outline in
    a few rows and then travels through the smooth interior, so this penalty
    moves seams into that interior (the painted sky on the heart) instead of
    keeping them out. ``retarget`` leaves the weight at zero.
    """
    gray = np.clip(np.rint(lum * 255.0), 0, 255).astype(np.uint8)
    blurred = cv2.GaussianBlur(gray, (0, 0), 2.2)
    gx = cv2.Sobel(blurred, cv2.CV_32F, 1, 0, ksize=3)
    gy = cv2.Sobel(blurred, cv2.CV_32F, 0, 1, ksize=3)
    magnitude = np.sqrt(gx * gx + gy * gy).astype(np.float32)
    cutoff = float(np.quantile(magnitude, 0.90))
    magnitude[magnitude < cutoff] = 0.0
    kernel = np.ones((3, 3), np.uint8)
    magnitude = cv2.dilate(magnitude, kernel)
    return _normalize(magnitude)


def line_protection(lum: np.ndarray, seam_direction: str, detector: str = "hough") -> np.ndarray:
    """Line map in ``[0, 1]`` for one seam direction.

    ``detector`` is ``hough`` (Canny plus the probabilistic Hough transform)
    or ``lsd`` (OpenCV's line-segment detector on the same blurred gray
    image). A segment must be at least 30 pixels and at least 12 percent
    of the short side. At most 40 of the longest segments are kept, so brick
    texture does not fill the mask.

    ``seam_direction`` is the seam being removed, not the line orientation.
    A vertical seam changes width, so vertical structures are protected
    most: a segment at angle ``theta`` from the x-axis receives weight
    ``|sin theta|``. A horizontal seam receives ``|cos theta|``, which
    protects horizontal structures. A horizon that spans the frame must
```

```
    still be crossed by every vertical seam; this weight does not also
    penalize columns and poles for that forced crossing.
    """
    if seam_direction not in ("vertical", "horizontal"):
        raise ValueError("seam_direction must be 'vertical' or 'horizontal'")

    gray_u8 = np.clip(np.rint(lum * 255.0), 0, 255).astype(np.uint8)
    # Blur before the detector so brick texture and foliage do not fill the mask.
    blurred = cv2.GaussianBlur(gray_u8, (5, 5), 1.2)
    height, width = lum.shape
    min_length = max(30, int(round(0.12 * min(height, width))))
    segments = _line_segments(blurred, min_length, detector)
    protection = np.zeros((height, width), np.float32)
    if len(segments) == 0:
        return protection
    ranked = []
    for x1, y1, x2, y2 in segments:
        length = float(np.hypot(x2 - x1, y2 - y1))
        if length < min_length:
            continue
        ranked.append((length, int(round(x1)), int(round(y1)), int(round(x2)), int(round(y2))))
    ranked.sort(reverse=True)
    layer = np.zeros((height, width), np.float32)
    for _, x1, y1, x2, y2 in ranked[:40]:
        # theta is the segment angle from the +x axis. Vertical lines have
        # |sin theta| = 1; horizontal lines have |cos theta| = 1.
        theta = np.arctan2(float(y2 - y1), float(x2 - x1))
        if seam_direction == "vertical":
            weight = abs(np.sin(theta))
        else:
            weight = abs(np.cos(theta))
        if weight < 1e-3:
            continue
        layer.fill(0.0)
        cv2.line(layer, (x1, y1), (x2, y2), float(weight), thickness=2)
        protection = np.maximum(protection, layer)
    return protection


def extra_energy(saliency: np.ndarray, lines: np.ndarray, w_s: float, w_l: float,
                 curves: np.ndarray | None = None, w_c: float = 0.0) -> np.ndarray:
    """Saliency, oriented lines, and optional curve boundaries."""
    cost = w_s * saliency + w_l * lines
    if curves is not None and w_c != 0.0:
        cost = cost + w_c * curves
    return cost.astype(np.float32)


def _line_segments(blurred_u8: np.ndarray, min_length: int, detector: str) -> np.ndarray:
    if detector == "lsd":
        raw = cv2.createLineSegmentDetector().detect(blurred_u8)[0]
    elif detector == "hough":
        edges = cv2.Canny(blurred_u8, 80, 160, apertureSize=3)
        raw = cv2.HoughLinesP(
            edges,
            rho=1,
            theta=np.pi / 180.0,
            threshold=50,
            minLineLength=min_length,
            maxLineGap=10,
        )
    else:
        raise ValueError("detector must be 'hough' or 'lsd'")
    if raw is None or len(raw) == 0:
        return np.zeros((0, 4), np.float32)
    # Hough is (N, 1, 4) on OpenCV 4 and (N, 4) on OpenCV 5. LSD is (N, 4).
    return np.asarray(raw, dtype=np.float32).reshape(-1, 4)


def _normalize(values: np.ndarray) -> np.ndarray:
    values = values.astype(np.float32)
    low = float(values.min())
    high = float(values.max())
    if high - low < 1e-8:
        return np.zeros_like(values)
    return (values - low) / (high - low)
```

Listing 2. src/energy.py

```
"""Vertical-seam dynamic programs, removal, and insertion.

A horizontal seam is a vertical seam on the transposed image. The caller
owns that transpose so saliency and line maps stay aligned.

Forward energy follows Rubinstein, Shamir, and Avidan (SIGGRAPH 2008).
Backward energy follows Avidan and Shamir (SIGGRAPH 2007).
"""

from __future__ import annotations

import numpy as np

from .energy import extra_energy, gradient_magnitude, luminance


def backward_seam(energy: np.ndarray) -> tuple[np.ndarray, float]:
    """Minimum seam under the classic backward cost.

    ``M(i, j) = E(i, j) + min(M(i-1, j-1), M(i-1, j), M(i-1, j+1))``.
    Returns the seam as one column index per row, and the seam cost.
    """
    height, width = energy.shape
    cumulative = np.zeros((height, width), np.float64)
    step = np.zeros((height, width), np.int8)
    cumulative[0] = energy[0]
    from_left = np.empty(width, np.float64)
    from_right = np.empty(width, np.float64)

    for i in range(1, height):
        previous = cumulative[i - 1]
        from_left[0] = np.inf
        from_left[1:] = previous[:-1]
        from_right[-1] = np.inf
        from_right[:-1] = previous[1:]
        # 0 = from j-1, 1 = from j, 2 = from j+1. Up is the starting choice.
        which = np.ones(width, np.int8)
        best = previous + 0.0
        better = from_left < best
        best = np.where(better, from_left, best)
        which = np.where(better, np.int8(0), which)
        better = from_right < best
        best = np.where(better, from_right, best)
        which = np.where(better, np.int8(2), which)
        cumulative[i] = energy[i] + best
        step[i] = which - 1

    return _backtrack(step, cumulative)


def forward_seam(lum: np.ndarray, pixel_cost: np.ndarray) -> tuple[np.ndarray, float]:
    """Minimum seam under forward energy plus a per-pixel cost.

    Removing pixel ``(i, j)`` joins its left and right neighbors. The new
    edge, and the edge against the pixel the seam just left, is:

        C_U(i, j) = |I(i, j+1) - I(i, j-1)|
        C_L(i, j) = C_U(i, j) + |I(i-1, j) - I(i, j-1)|
        C_R(i, j) = C_U(i, j) + |I(i-1, j) - I(i, j+1)|

    ``C_L``, ``C_U``, and ``C_R`` are the costs of arriving from ``(i-1, j-1)``,
    ``(i-1, j)``, and ``(i-1, j+1)``. The stored cost adds ``pixel_cost``,
    which carries saliency and line protection. Out-of-range neighbors are
    clamped to the border pixel; a step that would leave the image is ``inf``.
    """
    height, width = lum.shape
    cumulative = np.zeros((height, width), np.float64)
    step = np.zeros((height, width), np.int8)
    cumulative[0] = pixel_cost[0]
    left = np.empty(width, np.float64)
    right = np.empty(width, np.float64)
    from_left = np.empty(width, np.float64)
    from_right = np.empty(width, np.float64)

    for i in range(1, height):
        row = lum[i]
        above = lum[i - 1]
        left[0] = row[0]
        left[1:] = row[:-1]
```

```
        right[-1] = row[-1]
        right[:-1] = row[1:]

        cost_up = np.abs(right - left)
        cost_left = cost_up + np.abs(above - left)
        cost_right = cost_up + np.abs(above - right)

        previous = cumulative[i - 1]
        from_left[0] = np.inf
        from_left[1:] = previous[:-1] + cost_left[1:]
        from_right[-1] = np.inf
        from_right[:-1] = previous[1:] + cost_right[:-1]
        best = previous + cost_up
        which = np.ones(width, np.int8)
        better = from_left < best
        best = np.where(better, from_left, best)
        which = np.where(better, np.int8(0), which)
        better = from_right < best
        best = np.where(better, from_right, best)
        which = np.where(better, np.int8(2), which)
        cumulative[i] = pixel_cost[i] + best
        step[i] = which - 1

    return _backtrack(step, cumulative)


def vertical_seam(image: np.ndarray, saliency: np.ndarray, lines: np.ndarray,
                  mode: str, w_s: float, w_l: float,
                  curves: np.ndarray | None = None, w_c: float = 0.0) -> tuple[np.ndarray, float]:
    """One vertical seam on ``image`` using the matching line map."""
    lum = luminance(image)
    if mode == "classic":
        return backward_seam(gradient_magnitude(lum))
    if mode != "forward":
        raise ValueError("mode must be 'classic' or 'forward'")
    pixel_cost = extra_energy(saliency, lines, w_s, w_l, curves, w_c)
    return forward_seam(lum, pixel_cost)


def remove_vertical_seam(array: np.ndarray, seam: np.ndarray) -> np.ndarray:
    """Drop one pixel per row. Works for ``HxW``, ``HxWxC``, and index maps."""
    height, width = array.shape[:2]
    keep = np.ones((height, width), dtype=bool)
    keep[np.arange(height), seam] = False
    if array.ndim == 2:
        return array[keep].reshape(height, width - 1)
    return array[keep].reshape(height, width - 1, array.shape[2])


def insert_vertical_seams(image: np.ndarray, seams: list[np.ndarray]) -> np.ndarray:
    """Duplicate each recorded seam pixel, averaged with its right neighbor.

    ``seams`` stores column indices into ``image`` itself (the picture before
    insertion), one array of length ``H`` per seam. Seams must not reuse a pixel.
    """
    if not seams:
        return image
    height, width = image.shape[:2]
    channels = 1 if image.ndim == 2 else image.shape[2]
    source = image.reshape(height, width, channels).astype(np.float32)
    order = np.sort(np.stack(seams, axis=1).astype(np.int32), axis=1)
    count = order.shape[1]
    output = np.empty((height, width + count, channels), np.float32)

    for row in range(height):
        cols = order[row]
        written = 0
        cursor = 0
        src_row = source[row]
        dst_row = output[row]
        for col in cols:
            run = int(col) - cursor + 1
            dst_row[written:written + run] = src_row[cursor:int(col) + 1]
            written += run
            neighbor = src_row[int(col) + 1] if int(col) + 1 < width else src_row[int(col)]
            dst_row[written] = 0.5 * (src_row[int(col)] + neighbor)
            written += 1
            cursor = int(col) + 1
        if cursor < width:
```

```
            dst_row[written:] = src_row[cursor:]
    result = np.clip(np.rint(output), 0, 255).astype(np.uint8)
    if image.ndim == 2:
        return result[:, :, 0]
    return result


def _backtrack(step: np.ndarray, cumulative: np.ndarray) -> tuple[np.ndarray, float]:
    height, width = step.shape
    seam = np.zeros(height, np.int32)
    seam[-1] = int(np.argmin(cumulative[-1]))
    cost = float(cumulative[-1, seam[-1]])
    for i in range(height - 1, 0, -1):
        seam[i - 1] = int(np.clip(seam[i] + int(step[i, seam[i]]), 0, width - 1))
    return seam, cost
```

Listing 3. src/seam.py

```
"""Uniform scaling and center-crop baselines."""

from __future__ import annotations

import cv2
import numpy as np


def scale_to(image: np.ndarray, target_h: int, target_w: int) -> np.ndarray:
    """Resize to the exact target. Area interpolation when shrinking."""
    height, width = image.shape[:2]
    shrinking = target_h <= height and target_w <= width
    interpolation = cv2.INTER_AREA if shrinking else cv2.INTER_LINEAR
    return cv2.resize(image, (target_w, target_h), interpolation=interpolation)


def center_crop_resize(image: np.ndarray, target_h: int, target_w: int) -> np.ndarray:
    """Crop the centered window with the target aspect, then resize to it."""
    height, width = image.shape[:2]
    target_aspect = target_w / float(target_h)
    aspect = width / float(height)
    if aspect > target_aspect:
        new_w = max(1, int(round(height * target_aspect)))
        x0 = max(0, (width - new_w) // 2)
        cropped = image[:, x0:x0 + new_w]
    else:
        new_h = max(1, int(round(width / target_aspect)))
        y0 = max(0, (height - new_h) // 2)
        cropped = image[y0:y0 + new_h, :]
    if cropped.size == 0:
        cropped = image
    return scale_to(cropped, target_h, target_w)


def limit_long_side(image: np.ndarray, max_side: int = 640) -> np.ndarray:
    """Shrink so the long side is at most ``max_side``. Never enlarges."""
    height, width = image.shape[:2]
    long_side = max(height, width)
    if long_side <= max_side:
        return np.ascontiguousarray(image)
    scale = max_side / float(long_side)
    new_w = max(2, int(round(width * scale)))
    new_h = max(2, int(round(height * scale)))
    return scale_to(image, new_h, new_w)
```

Listing 4. src/baselines.py

```
"""Retarget one RGB image to a target height and width.

Methods
-------
scale
    Uniform resize.
crop
    Center-crop to the target aspect, then resize.
classic
    Backward-energy seam carving (gradient magnitude only).
ours
    Forward-energy seam carving plus saliency and Hough line protection.
```

```
    ``weights`` is ``(w_s, w_l)``. ``(0, 0)`` is forward energy alone,
    ``(w_s, 0)`` drops the line term, and ``(0, w_l)`` drops saliency.
hybrid
    The same cost as ``ours``. Seams stop once a side reaches 75 percent
    of its original length. A uniform scale does the rest of a deeper cut.
"""

from __future__ import annotations

import numpy as np

from .baselines import center_crop_resize, scale_to
from .energy import curve_protection, line_protection, luminance, spectral_residual_saliency
from .metrics import accumulate_seam, empty_counts
from .seam import insert_vertical_seams, remove_vertical_seam, vertical_seam

DEFAULT_WEIGHTS = (0.5, 0.8)
# Seams are allowed to remove at most this fraction of a side.
HYBRID_KEEP = 0.75


def retarget(image: np.ndarray, target_hw: tuple[int, int], method: str = "ours",
             weights: tuple[float, float] = DEFAULT_WEIGHTS, return_info: bool = False):
    """Return the retargeted RGB uint8 image.

    When ``return_info`` is true, also return a dict of seam statistics.
    Statistics are meaningful for ``classic``, ``ours``, and the carved
    part of ``hybrid``. Scale and crop report an empty seam count.
    Hybrid line and saliency scores cover only the seams, not the scale
    that finishes a reduction past 75 percent.
    """
    rgb = _as_uint8_rgb(image)
    target_h = max(2, int(target_hw[0]))
    target_w = max(2, int(target_hw[1]))
    w_s, w_l = float(weights[0]), float(weights[1])

    if method == "scale":
        out = scale_to(rgb, target_h, target_w)
        info = _baseline_info(out, (target_h, target_w))
        return (out, info) if return_info else out
    if method == "crop":
        out = center_crop_resize(rgb, target_h, target_w)
        info = _baseline_info(out, (target_h, target_w))
        return (out, info) if return_info else out
    if method not in ("classic", "ours", "hybrid"):
        raise ValueError("method must be 'scale', 'crop', 'classic', 'ours', or 'hybrid'")

    mode = "classic" if method == "classic" else "forward"
    height, width = rgb.shape[:2]
    if method == "hybrid":
        # Same cost as ours. Seams stop at 75% of the original side.
        shrink_h = _hybrid_stop(height, target_h)
        shrink_w = _hybrid_stop(width, target_w)
        detector, w_c = "hough", 0.0
    else:
        shrink_h, shrink_w = min(target_h, height), min(target_w, width)
        detector, w_c = "hough", 0.0
    carved, counts = _shrink(rgb, shrink_h, shrink_w, mode, w_s, w_l, w_c, detector)
    counts["scaled_after_seams"] = 0
    scale_h = target_h if target_h < carved.shape[0] else carved.shape[0]
    scale_w = target_w if target_w < carved.shape[1] else carved.shape[1]
    if (scale_h, scale_w) != carved.shape[:2]:
        carved = scale_to(carved, scale_h, scale_w)
        counts["scaled_after_seams"] = 1

    if target_w > carved.shape[1]:
        carved, insert_counts = _enlarge_width(carved, target_w, mode, w_s, w_l, w_c, detector)
        _merge_path_counts(counts, insert_counts)
    if target_h > carved.shape[0]:
        carved, insert_counts = _enlarge_height(carved, target_h, mode, w_s, w_l, w_c, detector)
        _merge_path_counts(counts, insert_counts)

    info = dict(counts)
    info["output_hw"] = tuple(carved.shape[:2])
    info["target_hw"] = (target_h, target_w)
    return (carved, info) if return_info else carved


def next_vertical_seam(image: np.ndarray, method: str = "ours",
```

```
                      weights: tuple[float, float] = DEFAULT_WEIGHTS) -> np.ndarray:
    """Column index per row of the next vertical seam, for the demo overlay."""
    rgb = _as_uint8_rgb(image)
    maps = protection_maps(rgb)
    mode = "classic" if method == "classic" else "forward"
    w_s, w_l = (0.0, 0.0) if method == "classic" else (float(weights[0]), float(weights[1]))
    seam, _ = vertical_seam(rgb, maps["saliency"], maps["lines_vertical"], mode, w_s, w_l)
    return seam


def protection_maps(image: np.ndarray) -> dict[str, np.ndarray]:
    """Luminance, saliency, and both line maps, for the demo overlays."""
    rgb = _as_uint8_rgb(image)
    lum = luminance(rgb)
    return {
        "luminance": lum,
        "saliency": spectral_residual_saliency(lum),
        "lines_vertical": line_protection(lum, "vertical"),
        "lines_horizontal": line_protection(lum, "horizontal"),
    }


def _hybrid_stop(original: int, target: int, keep: float = HYBRID_KEEP) -> int:
    """Size to reach with seams. A deeper cut is finished by scaling."""
    if target >= original:
        return original
    floor = max(2, int(round(original * keep)))
    return max(target, floor)


def _drop_curve(curves, seam, transpose: bool):
    if curves is None:
        return None
    if transpose:
        return remove_vertical_seam(curves.T, seam).T
    return remove_vertical_seam(curves, seam)


def _shrink(image, target_h, target_w, mode, w_s, w_l, w_c=0.0, detector="hough"):
    lum = luminance(image)
    saliency = spectral_residual_saliency(lum)
    line_v = line_protection(lum, "vertical", detector)
    line_h = line_protection(lum, "horizontal", detector)
    curves = curve_protection(lum) if w_c else None
    counts = empty_counts()
    counts["saliency_total"] = float(saliency.sum())

    current = image
    while current.shape[0] > target_h or current.shape[1] > target_w:
        can_v = current.shape[1] > target_w and current.shape[1] > 2
        can_h = current.shape[0] > target_h and current.shape[0] > 2
        if not can_v and not can_h:
            break
        if can_v and can_h:
            seam_v, cost_v = vertical_seam(current, saliency, line_v, mode, w_s, w_l, curves, w_c)
            seam_h, cost_h = _horizontal_seam(current, saliency, line_h, mode, w_s, w_l, curves, w_c)
            take_vertical = cost_v <= cost_h
        else:
            take_vertical = can_v
            seam_v = seam_h = None

        if take_vertical:
            if seam_v is None:
                seam_v, _ = vertical_seam(current, saliency, line_v, mode, w_s, w_l, curves, w_c)
            accumulate_seam(counts, saliency, line_v, seam_v, vertical=True)
            current = remove_vertical_seam(current, seam_v)
            saliency = remove_vertical_seam(saliency, seam_v)
            line_v = remove_vertical_seam(line_v, seam_v)
            line_h = remove_vertical_seam(line_h, seam_v)
            curves = _drop_curve(curves, seam_v, transpose=False)
        else:
            if seam_h is None:
                seam_h, _ = _horizontal_seam(current, saliency, line_h, mode, w_s, w_l, curves, w_c)
            transposed_sal = saliency.T
            transposed_line = line_h.T
            accumulate_seam(counts, transposed_sal, transposed_line, seam_h, vertical=False)
            current = _transpose_image(remove_vertical_seam(_transpose_image(current), seam_h))
            saliency = remove_vertical_seam(transposed_sal, seam_h).T
            line_h = remove_vertical_seam(transposed_line, seam_h).T
```

```
            line_v = remove_vertical_seam(line_v.T, seam_h).T
            curves = _drop_curve(curves, seam_h, transpose=True)
    return current, counts


def _enlarge_width(image, target_w, mode, w_s, w_l, w_c=0.0, detector="hough"):
    """Record seams by removal, then insert averaged duplicates into ``image``."""
    current = image
    total = empty_counts()
    while current.shape[1] < target_w:
        room = current.shape[1] - 2
        need = target_w - current.shape[1]
        step = min(need, room)
        if step <= 0:
            break
        seams, counts = _record_vertical_seams(current, step, mode, w_s, w_l, w_c, detector)
        _merge_path_counts(total, counts)
        current = insert_vertical_seams(current, seams)
    return current, total


def _enlarge_height(image, target_h, mode, w_s, w_l, w_c=0.0, detector="hough"):
    transposed = _transpose_image(image)
    enlarged, counts = _enlarge_width(transposed, target_h, mode, w_s, w_l, w_c, detector)
    counts["horizontal_seams"] = counts["vertical_seams"]
    counts["vertical_seams"] = 0
    return _transpose_image(enlarged), counts


def _record_vertical_seams(image, count, mode, w_s, w_l, w_c=0.0, detector="hough"):
    height, width = image.shape[:2]
    lum = luminance(image)
    saliency = spectral_residual_saliency(lum)
    lines = line_protection(lum, "vertical", detector)
    curves = curve_protection(lum) if w_c else None
    columns = np.tile(np.arange(width, dtype=np.int32), (height, 1))
    work = image
    seams = []
    counts = empty_counts()
    counts["saliency_total"] = float(saliency.sum())
    for _ in range(count):
        seam, _ = vertical_seam(work, saliency, lines, mode, w_s, w_l, curves, w_c)
        rows = np.arange(work.shape[0])
        seams.append(columns[rows, seam].copy())
        # Path statistics follow the chosen seam. Saliency is not marked as
        # removed: insertion keeps those pixels and adds a duplicate.
        counts["line_hits"] += int(np.count_nonzero(lines[rows, seam] > 0))
        counts["seam_pixels"] += int(seam.shape[0])
        counts["vertical_seams"] += 1
        work = remove_vertical_seam(work, seam)
        saliency = remove_vertical_seam(saliency, seam)
        lines = remove_vertical_seam(lines, seam)
        curves = _drop_curve(curves, seam, transpose=False)
        columns = remove_vertical_seam(columns, seam)
    return seams, counts


def _horizontal_seam(image, saliency, line_h, mode, w_s, w_l, curves=None, w_c=0.0):
    curve_t = None if curves is None else curves.T
    return vertical_seam(
        _transpose_image(image), saliency.T, line_h.T, mode, w_s, w_l, curve_t, w_c,
    )


def _transpose_image(image: np.ndarray) -> np.ndarray:
    if image.ndim == 2:
        return np.ascontiguousarray(np.transpose(image, (1, 0)))
    return np.ascontiguousarray(np.transpose(image, (1, 0, 2)))


def _merge_path_counts(total: dict, extra: dict) -> None:
    for key in ("line_hits", "seam_pixels", "vertical_seams", "horizontal_seams"):
        total[key] += extra[key]


def _baseline_info(image, target_hw) -> dict:
    counts = empty_counts()
    counts["output_hw"] = tuple(image.shape[:2])
    counts["target_hw"] = target_hw
```

```
    return counts


def __as__uint8__rgb(image: np.ndarray) -> np.ndarray:
    array = np.asarray(image)
    if array.ndim == 2:
        array = np.stack([array, array, array], axis=-1)
    if array.shape[2] == 4:
        array = array[:, :, :3]
    if array.dtype != np.uint8:
        values = array.astype(np.float32)
        if values.max() <= 1.0:
            values = values * 255.0
        array = np.clip(np.rint(values), 0, 255).astype(np.uint8)
    return np.ascontiguousarray(array)
```

Listing 5. src/retarget.py

```
"""Scores collected while seams are chosen.

Saliency mass is the sum of the spectral-residual map on pixels that are
actually deleted. Line crossings count seam pixels that land on a positive
line-protection value, reported per 100 seam pixels. Scale and crop do not
choose seams, so those two scores are left unset for them.
"""

from __future__ import annotations

import numpy as np


def empty__counts() -> dict:
    return {
        "saliency__total": 0.0,
        "saliency__removed": 0.0,
        "line__hits": 0,
        "seam__pixels": 0,
        "vertical__seams": 0,
        "horizontal__seams": 0,
    }


def saliency__removed__fraction(counts: dict) -> float:
    total = float(counts["saliency__total"])
    if total <= 0.0:
        return 0.0
    return float(counts["saliency__removed"]) / total


def line__crossings__per__100(counts: dict) -> float:
    pixels = int(counts["seam__pixels"])
    if pixels <= 0:
        return 0.0
    return 100.0 * float(counts["line__hits"]) / pixels


def size__matches(image: np.ndarray, target__hw: tuple[int, int]) -> bool:
    return tuple(image.shape[:2]) == (int(target__hw[0]), int(target__hw[1]))


def accumulate__seam(counts: dict, saliency: np.ndarray, lines: np.ndarray,
                     seam: np.ndarray, vertical: bool) -> None:
    rows = np.arange(seam.shape[0])
    counts["saliency__removed"] += float(saliency[rows, seam].sum())
    counts["line__hits"] += int(np.count__nonzero(lines[rows, seam] > 0))
    counts["seam__pixels"] += int(seam.shape[0])
    if vertical:
        counts["vertical__seams"] += 1
    else:
        counts["horizontal__seams"] += 1
```

Listing 6. src/metrics.py

```
"""RGB overlays for the demo and the experiment figures."""

from __future__ import annotations

```

```
import cv2
import numpy as np


def colorize(values: np.ndarray) -> np.ndarray:
    """Map a ``[0, 1]`` array to an RGB image with the viridis colormap."""
    scaled = np.clip(np.rint(values * 255.0), 0, 255).astype(np.uint8)
    bgr = cv2.applyColorMap(scaled, cv2.COLORMAP_VIRIDIS)
    return cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)


def draw_vertical_seam(image: np.ndarray, seam: np.ndarray, color=(220, 40, 40)) -> np.ndarray:
    """Paint a vertical seam, two pixels wide, on a copy of ``image``."""
    painted = image.copy()
    height, width = painted.shape[:2]
    rows = np.arange(height)
    cols = np.clip(seam, 0, width - 1)
    painted[rows, cols] = color
    painted[rows, np.clip(cols - 1, 0, width - 1)] = color
    return painted
```

Listing 7. src/display.py

```
"""Interactive demo for spatial-domain image retargeting.

Upload an image, pick a target size as a percent of the working resolution,
and compare uniform scaling, center-crop, classic seam carving, the
combined forward energy, and a hybrid that stops seams at 75% width
and scales the rest. The long side is capped at 640 px so the dynamic
program stays responsive. Weights w_s and w_l are the saliency and
line-protection multipliers. Hybrid uses that same cost, stops seams
at 75% of the original side, and scales anything beyond that.
"""

from __future__ import annotations

import time
from pathlib import Path

import gradio as gr
import numpy as np

from src.baselines import limit_long_side
from src.display import colorize, draw_vertical_seam
from src.metrics import line_crossings_per_100, saliency_removed_fraction
from src.retarget import next_vertical_seam, protection_maps, retarget

ROOT = Path(__file__).resolve().parent
IMAGE_DIR = ROOT / "experiments" / "retargetme"
ROLES_PATH = ROOT / "experiments" / "results" / "roles.txt"
MAX_SIDE = 640


def _find_named(stem: str) -> Path | None:
    for suffix in (".png", ".jpg", ".jpeg", ".PNG", ".JPG"):
        candidate = IMAGE_DIR / f"{stem}{suffix}"
        if candidate.exists():
            return candidate
    return None


def example_paths() -> list[str]:
    """Prefer the line, saliency, and failure images chosen by the experiments."""
    ordered: list[Path] = []
    if ROLES_PATH.exists():
        roles = {}
        for line in ROLES_PATH.read_text(encoding="utf-8").splitlines():
            if "=" in line:
                key, value = line.split("=", 1)
                roles[key.strip()] = value.strip()
        for key in ("lines", "saliency", "failure"):
            path = _find_named(roles.get(key, ""))
            if path is not None:
                ordered.append(path)
    if not ordered and IMAGE_DIR.exists():
        ordered = [
            path for path in sorted(IMAGE_DIR.iterdir())
            if path.suffix.lower() in {".png", ".jpg", ".jpeg"}
        ][:3]
```

```
    return [str(path) for path in ordered]


def _fit_height(image: np.ndarray, height: int) -> np.ndarray:
    scale = height / float(image.shape[0])
    width = max(1, int(round(image.shape[1] * scale)))
    import cv2
    return cv2.resize(image, (width, height), interpolation=cv2.INTER_AREA)


def _hstack(images: list[np.ndarray]) -> np.ndarray:
    height = max(image.shape[0] for image in images)
    fitted = [_fit_height(image, height) for image in images]
    gap = np.full((height, 8, 3), 255, np.uint8)
    pieces: list[np.ndarray] = []
    for index, image in enumerate(fitted):
        if index:
            pieces.append(gap)
        pieces.append(image)
    return np.hstack(pieces)


def retarget_image(image, width_percent, height_percent, method, w_s, w_l,
                   show_saliency, show_lines, show_seam):
    if image is None:
        return None, None, "Upload an image or pick an example."
    rgb = limit_long_side(np.asarray(image), MAX_SIDE)
    height, width = rgb.shape[:2]
    target_h = max(2, int(round(height * float(height_percent) / 100.0)))
    target_w = max(2, int(round(width * float(width_percent) / 100.0)))
    started = time.perf_counter()
    output, info = retarget(
        rgb, (target_h, target_w), method=method, weights=(float(w_s), float(w_l)), return_info=True,
    )
    elapsed = time.perf_counter() - started
    pair = _hstack([rgb, output])

    overlays = []
    if show_saliency or show_lines or show_seam:
        maps = protection_maps(rgb)
        if show_saliency:
            overlays.append(colorize(maps["saliency"]))
        if show_lines:
            overlays.append(colorize(maps["lines_vertical"]))
        if show_seam and method in ("classic", "ours", "hybrid"):
            seam = next_vertical_seam(rgb, method=method, weights=(float(w_s), float(w_l)))
            overlays.append(draw_vertical_seam(rgb, seam))
    overlay_image = _hstack(overlays) if overlays else None

    message = (
        f"Working size {height} x {width} -> {output.shape[0]} x {output.shape[1]} "
        f"in {elapsed:.1f}s. Method: {method}."
    )
    if method in ("classic", "ours", "hybrid") and info.get("seam_pixels", 0) > 0:
        message += (
            f" Saliency mass removed: {saliency_removed_fraction(info):.3f}."
            f" Line pixels per 100 seam pixels: {line_crossings_per_100(info):.1f}."
        )
    if info.get("scaled_after_seams"):
        message += " Seams stopped at 75% of the original side; scaling finished the cut."
    return pair, overlay_image, message


def build() -> gr.Blocks:
    with gr.Blocks(title="Image retargeting") as demo:
        gr.Markdown(
            """
# Image retargeting

Spatial-domain seam carving. **Ours** adds spectral-residual saliency and an
orientation-aware line map to Rubinstein forward energy. **Hybrid** uses
that same cost, carves only down to 75% of the original side, and scales the rest.

Suggested demo: a facade (turn the line weight up), a person (saliency),
then San Francisco at 50% width, comparing ours with hybrid.
            """
        )
        with gr.Row():
            source = gr.Image(type="numpy", label="Input", height=360)
```

```
            with gr.Column():
                width_percent = gr.Slider(50, 150, value=75, step=1, label="Target width (%)")
                height_percent = gr.Slider(50, 150, value=100, step=1, label="Target height (%)")
                method = gr.Radio(
                    ["hybrid", "ours", "classic", "scale", "crop"],
                    value="hybrid",
                    label="Method",
                )
                w_s = gr.Slider(0.0, 2.0, value=0.5, step=0.1, label="Saliency weight w_s")
                w_l = gr.Slider(0.0, 2.0, value=0.8, step=0.1, label="Line weight w_l")
                show_saliency = gr.Checkbox(value=True, label="Show saliency")
                show_lines = gr.Checkbox(value=True, label="Show vertical-seam line map")
                show_seam = gr.Checkbox(value=True, label="Show the next vertical seam")
                run = gr.Button("Retarget", variant="primary")
        result = gr.Image(type="numpy", label="Original (left) and retargeted (right)")
        maps = gr.Image(type="numpy", label="Saliency, line map, next seam")
        status = gr.Textbox(label="Notes", lines=2)
        run.click(
            retarget_image,
            inputs=[source, width_percent, height_percent, method, w_s, w_l,
                    show_saliency, show_lines, show_seam],
            outputs=[result, maps, status],
        )
        paths = example_paths()
        if paths:
            gr.Examples(examples=[[path] for path in paths], inputs=[source], label="Examples")
    return demo


if __name__ == "__main__":
    build().launch()
```

Listing 8. app.py

```
"""Download the RetargetMe source-image archive and keep a small subset.

The archive is the public "source images only" zip linked from
http://people.csail.mit.edu/mrub/retargetme/download.html
Full images are not committed; this script fetches them.
"""

from __future__ import annotations

import re
import sys
import urllib.request
import zipfile
from pathlib import Path

PAGE = "https://people.csail.mit.edu/mrub/retargetme/download.html"
_OPENER = urllib.request.build_opener()
_OPENER.addheaders = [("User-Agent", "Mozilla/5.0 (compatible; ee604-retarget/1.0)")]
urllib.request.install_opener(_OPENER)
ROOT = Path(__file__).resolve().parents[1]
DEST = ROOT / "experiments" / "retargetme"
ZIP_PATH = ROOT / "experiments" / "retargetme_source.zip"

# Eight source images: people, buildings, an animal, an object, a landscape,
# and a street scene. Names are the stems published in the RetargetMe archive.
KEEP = (
    "obama.png",
    "child.png",
    "brick_house.png",
    "Johanneskirche.png",
    "eagle.png",
    "butterfly.png",
    "waterfall.png",
    "Sanfrancisco.png",
)


def _source_zip_url() -> str:
    with urllib.request.urlopen(PAGE, timeout=60) as response:
        html = response.read().decode("utf-8", "replace")
    anchors = re.findall(r'<a[^>]+href="([^"]+)"[^>]*>(.*?)</a>', html, flags=re.I | re.S)
    candidates = []
    for href, raw_text in anchors:
        text = re.sub(r"<[^>]+>", "", raw_text)
        text = re.sub(r"\s+", " ", text).strip().lower()
```

```
        if href.lower().endswith(".zip"):
            candidates.append((text, urllib.request.urljoin(PAGE, href)))
            print(f"zip link: {text} -> {candidates[-1][1]}")
    for text, url in candidates:
        if "source image" in text and "only" in text:
            return url
    for text, url in candidates:
        if "source" in text and "retarget" not in text:
            return url
    raise RuntimeError("Could not find the source-image zip on the RetargetMe download page")


def main() -> None:
    DEST.mkdir(parents=True, exist_ok=True)
    present = {path.name for path in DEST.iterdir() if path.is_file()}
    if set(KEEP).issubset(present):
        print(f"subset already in {DEST}")
        return
    if not ZIP_PATH.exists():
        url = _source_zip_url()
        print(f"downloading {url}")
        urllib.request.urlretrieve(url, ZIP_PATH)
    with zipfile.ZipFile(ZIP_PATH) as archive:
        by_name = {Path(name).name: name for name in archive.namelist()}
        missing = [name for name in KEEP if name not in by_name]
        if missing:
            raise RuntimeError(f"missing from archive: {missing}")
        for stale in list(DEST.iterdir()):
            if stale.is_file() and stale.name not in KEEP:
                stale.unlink()
        for name in KEEP:
            (DEST / name).write_bytes(archive.read(by_name[name]))
            print("kept", name)
    print(f"wrote {len(KEEP)} images to {DEST}")


if __name__ == "__main__":
    try:
        main()
    except Exception as exc:
        print(f"download failed: {exc}", file=sys.stderr)
        sys.exit(1)
```

Listing 9. experiments/download_retargetme.py

```
"""Compare scale, crop, classic seams, and the combined energy.

Writes a CSV, a text summary, and OpenCV contact sheets. Images are the
RetargetMe subset in ``experiments/retargetme``. The long side is capped
at 640 px before any method runs.
"""

from __future__ import annotations

import csv
import sys
import time
from pathlib import Path

import cv2
import numpy as np

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from src.baselines import limit_long_side  # noqa: E402
from src.display import colorize, draw_vertical_seam  # noqa: E402
from src.energy import line_protection, luminance  # noqa: E402
from src.metrics import line_crossings_per_100, saliency_removed_fraction, size_matches  # noqa: E402
from src.retarget import next_vertical_seam, protection_maps, retarget  # noqa: E402

IMAGE_DIR = ROOT / "experiments" / "retargetme"
RESULT_DIR = ROOT / "experiments" / "results"
MAX_SIDE = 640
PANEL_H = 168
WEIGHTS = {
    "forward": (0.0, 0.0),
    "forward_saliency": (0.5, 0.0),
    "forward_lines": (0.0, 0.8),
```

```
    "ours": (0.5, 0.8),
    "classic": (0.0, 0.0),
    "scale": (0.0, 0.0),
    "crop": (0.0, 0.0),
}
SEAM_METHODS = {"classic", "ours", "forward", "forward_saliency", "forward_lines"}


def load_rgb(path: Path) -> np.ndarray:
    bgr = cv2.imread(str(path), cv2.IMREAD_COLOR)
    if bgr is None:
        raise FileNotFoundError(path)
    return cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)


def discover_images() -> list[Path]:
    paths = sorted(
        [p for p in IMAGE_DIR.iterdir() if p.suffix.lower() in {".png", ".jpg", ".jpeg"}]
    )
    if len(paths) < 1:
        raise FileNotFoundError(
            f"No images in {IMAGE_DIR}. Run experiments/download_retargetme.py first."
        )
    return paths


def run_one(image: np.ndarray, target_hw, method: str) -> tuple[np.ndarray, dict, float]:
    weights = WEIGHTS[method]
    started = time.perf_counter()
    if method in ("forward", "forward_saliency", "forward_lines"):
        output, info = retarget(image, target_hw, method="ours", weights=weights, return_info=True)
    else:
        output, info = retarget(image, target_hw, method=method, weights=weights, return_info=True)
    elapsed = time.perf_counter() - started
    return output, info, elapsed


def row_for(name, setting, method, image, output, info, elapsed, target_hw) -> dict:
    seam = method in SEAM_METHODS or method in ("forward", "forward_saliency", "forward_lines")
    sal = ""
    lines = ""
    if seam and info.get("seam_pixels", 0) > 0:
        sal = f"{saliency_removed_fraction(info):.6f}"
        lines = f"{line_crossings_per_100(info):.4f}"
    elif seam:
        sal = "0"
        lines = "0"
    return {
        "image": name,
        "setting": setting,
        "method": method,
        "w_s": WEIGHTS[method][0],
        "w_l": WEIGHTS[method][1],
        "target_h": target_hw[0],
        "target_w": target_hw[1],
        "out_h": output.shape[0],
        "out_w": output.shape[1],
        "size_ok": int(size_matches(output, target_hw)),
        "saliency_removed_fraction": sal,
        "line_crossings_per_100": lines,
        "runtime_s": f"{elapsed:.4f}",
    }


def fit_panel(image: np.ndarray, title: str) -> np.ndarray:
    height, width = image.shape[:2]
    new_w = max(1, int(round(width * (PANEL_H / float(height)))))
    resized = cv2.resize(image, (new_w, PANEL_H), interpolation=cv2.INTER_AREA)
    bar = np.full((26, new_w, 3), 255, np.uint8)
    cv2.putText(bar, title, (4, 18), cv2.FONT_HERSHEY_SIMPLEX, 0.48, (0, 0, 0), 1, cv2.LINE_AA)
    return np.vstack([bar, resized])


def stack_rows(rows: list[np.ndarray]) -> np.ndarray:
    width = max(row.shape[1] for row in rows)
    padded = []
    for row in rows:
        if row.shape[1] == width:
            padded.append(row)
```

```
            continue
        canvas = np.full((row.shape[0], width, 3), 255, np.uint8)
        canvas[:, :row.shape[1]] = row
        padded.append(canvas)
    return np.vstack(padded)


def save_rgb(path: Path, image: np.ndarray) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    bgr = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
    cv2.imwrite(str(path), bgr)


def contact_sheet(records: list[tuple[str, list[tuple[str, np.ndarray]]]]) -> np.ndarray:
    """records: (row title, [(panel title, image), ...])."""
    rows = []
    for row_title, panels in records:
        tiles = [fit_panel(image, title) for title, image in panels]
        # A narrow name tag on the left of the row.
        tag_h = tiles[0].shape[0]
        tag = np.full((tag_h, 150, 3), 245, np.uint8)
        cv2.putText(tag, row_title[:18], (6, tag_h // 2), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 0, 0), 1, cv2.LINE_AA)
        body = np.hstack(tiles)
        if tag.shape[0] != body.shape[0]:
            tag = cv2.resize(tag, (tag.shape[1], body.shape[0]))
        rows.append(np.hstack([tag, body]))
    return stack_rows(rows)


def choose_roles(images: dict[str, np.ndarray]) -> dict[str, str]:
    """Line, saliency, and failure examples used in the paper and the demo.

    Johanneskirche has long vertical mullions, obama is the person, and
    Sanfrancisco is the busy scene. Other names are used when that subset
    is not the one on disk.
    """
    preferred = {
        "lines": "Johanneskirche",
        "saliency": "obama",
        "failure": "Sanfrancisco",
        "height": "waterfall",
        "enlarge": "butterfly",
    }
    if all(name in images for name in preferred.values()):
        return preferred
    line_score = {}
    saliency_peak = {}
    edge_fraction = {}
    for name, image in images.items():
        lum = luminance(image)
        lines = line_protection(lum, "vertical")
        sal = protection_maps(image)["saliency"]
        edges = cv2.Canny((lum * 255).astype(np.uint8), 80, 160)
        line_score[name] = float((lines > 0.75).mean())
        saliency_peak[name] = float(sal.max() / (sal.mean() + 1e-6))
        edge_fraction[name] = float(np.count_nonzero(edges)) / edges.size
    names = list(images)
    return {
        "lines": max(line_score, key=line_score.get),
        "saliency": max(saliency_peak, key=saliency_peak.get),
        "failure": max(edge_fraction, key=edge_fraction.get),
        "height": names[0],
        "enlarge": names[min(1, len(names) - 1)],
    }


def write_summary(rows: list[dict], path: Path) -> None:
    groups: dict[tuple[str, str], list[dict]] = {}
    for row in rows:
        groups.setdefault((row["setting"], row["method"]), []).append(row)
    lines = ["setting,method,n,mean_saliency_removed,mean_line_crossings_per_100,mean_runtime_s,all_sizes_ok"]
    for key in sorted(groups):
        group = groups[key]
        sal_vals = [float(r["saliency_removed_fraction"]) for r in group if r["saliency_removed_fraction"] != ""]
        line_vals = [float(r["line_crossings_per_100"]) for r in group if r["line_crossings_per_100"] != ""]
        run_vals = [float(r["runtime_s"]) for r in group]
        sal = sum(sal_vals) / len(sal_vals) if sal_vals else float("nan")
        crossed = sum(line_vals) / len(line_vals) if line_vals else float("nan")
        runtime = sum(run_vals) / len(run_vals)
```

```
        sizes_ok = all(int(r["size_ok"]) == 1 for r in group)
        lines.append(
            f"{key[0]},{key[1]},{len(group)},{sal:.6f},{crossed:.4f},{runtime:.4f},{int(sizes_ok)}"
        )
    path.write_text("\n".join(lines) + "\n", encoding="utf-8")


def main() -> None:
    RESULT_DIR.mkdir(parents=True, exist_ok=True)
    paths = discover_images()
    images = {path.stem: limit_long_side(load_rgb(path), MAX_SIDE) for path in paths}
    roles = choose_roles(images)
    (RESULT_DIR / "roles.txt").write_text(
        "\n".join(f"{role}={name}" for role, name in roles.items()) + "\n",
        encoding="utf-8",
    )
    print("roles", roles, flush=True)

    rows: list[dict] = []
    outputs: dict[tuple[str, str, str], np.ndarray] = {}

    width_75_methods = ["scale", "crop", "classic", "forward", "forward_saliency", "forward_lines", "ours"]
    width_50_methods = ["scale", "crop", "classic", "ours"]

    for name, image in images.items():
        height, width = image.shape[:2]
        jobs = [("width_75", (height, max(2, int(round(width * 0.75)))), width_75_methods)]
        jobs.append(("width_50", (height, max(2, int(round(width * 0.50)))), width_50_methods))
        if name == roles["height"]:
            jobs.append(("height_75", (max(2, int(round(height * 0.75))), width), ["scale", "classic", "ours"]))
        if name == roles["enlarge"]:
            jobs.append(("enlarge_150", (height, max(2, int(round(width * 1.50)))), ["scale", "ours"]))
        for setting, target, methods in jobs:
            for method in methods:
                print(f"{name} {setting} {method}", flush=True)
                output, info, elapsed = run_one(image, target, method)
                rows.append(row_for(name, setting, method, image, output, info, elapsed, target))
                outputs[(name, setting, method)] = output
                print(f"  {elapsed:.1f}s -> {output.shape[0]}x{output.shape[1]}", flush=True)

    csv_path = RESULT_DIR / "metrics.csv"
    with csv_path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)
    write_summary(rows, RESULT_DIR / "summary.csv")

    compare_75 = []
    compare_50 = []
    for name, image in images.items():
        compare_75.append((name, [
            ("Original", image),
            ("Scale", outputs[(name, "width_75", "scale")]),
            ("Crop", outputs[(name, "width_75", "crop")]),
            ("Classic", outputs[(name, "width_75", "classic")]),
            ("Ours", outputs[(name, "width_75", "ours")]),
        ]))
        compare_50.append((name, [
            ("Original", image),
            ("Scale", outputs[(name, "width_50", "scale")]),
            ("Crop", outputs[(name, "width_50", "crop")]),
            ("Classic", outputs[(name, "width_50", "classic")]),
            ("Ours", outputs[(name, "width_50", "ours")]),
        ]))
    save_rgb(RESULT_DIR / "fig_compare_75.png", contact_sheet(compare_75))
    save_rgb(RESULT_DIR / "fig_compare_50.png", contact_sheet(compare_50))
    paper_names = [roles["lines"], roles["saliency"], roles["failure"], roles["height"]]
    by_name_75 = {name: row for name, row in compare_75}
    by_name_50 = {name: row for name, row in compare_50}
    save_rgb(
        RESULT_DIR / "fig_compare_75_paper.png",
        contact_sheet([(name, by_name_75[name]) for name in paper_names]),
    )
    save_rgb(
        RESULT_DIR / "fig_compare_50_paper.png",
        contact_sheet([(name, by_name_50[name]) for name in paper_names]),
    )

    height_name = roles["height"]
```

```
    height_image = images[height_name]
    save_rgb(RESULT_DIR / "fig_height.png", contact_sheet([(height_name, [
        ("Original", height_image),
        ("Scale", outputs[(height_name, "height_75", "scale")]),
        ("Classic", outputs[(height_name, "height_75", "classic")]),
        ("Ours", outputs[(height_name, "height_75", "ours")]),
    ])]))

    enlarge_name = roles["enlarge"]
    enlarge_image = images[enlarge_name]
    save_rgb(RESULT_DIR / "fig_enlarge.png", contact_sheet([(enlarge_name, [
        ("Original", enlarge_image),
        ("Scale", outputs[(enlarge_name, "enlarge_150", "scale")]),
        ("Ours", outputs[(enlarge_name, "enlarge_150", "ours")]),
    ])]))

    failure_name = roles["failure"]
    save_rgb(RESULT_DIR / "fig_failure.png", contact_sheet([(failure_name + " 50%", [
        ("Original", images[failure_name]),
        ("Scale", outputs[(failure_name, "width_50", "scale")]),
        ("Crop", outputs[(failure_name, "width_50", "crop")]),
        ("Classic", outputs[(failure_name, "width_50", "classic")]),
        ("Ours", outputs[(failure_name, "width_50", "ours")]),
    ])]))

    line_name = roles["lines"]
    line_image = images[line_name]
    save_rgb(RESULT_DIR / "fig_ablation.png", contact_sheet([(line_name, [
        ("Forward", outputs[(line_name, "width_75", "forward")]),
        ("+Saliency", outputs[(line_name, "width_75", "forward_saliency")]),
        ("+Lines", outputs[(line_name, "width_75", "forward_lines")]),
        ("Full", outputs[(line_name, "width_75", "ours")]),
        ("Classic", outputs[(line_name, "width_75", "classic")]),
    ])]))

    maps = protection_maps(line_image)
    seam = next_vertical_seam(line_image, method="ours", weights=WEIGHTS["ours"])
    save_rgb(RESULT_DIR / "fig_maps.png", contact_sheet([(line_name, [
        ("Input", line_image),
        ("Saliency", colorize(maps["saliency"])),
        ("Lines", colorize(maps["lines_vertical"])),
        ("Next seam", draw_vertical_seam(line_image, seam)),
    ])]))
    print("wrote", RESULT_DIR, flush=True)


if __name__ == "__main__":
    main()
```

Listing 10. experiments/run_experiments.py

```
"""50% width: full seam carve versus seams down to 75%, then scale.

Also checks that a 75% target makes hybrid identical to the full model.
"""

from __future__ import annotations

import csv
import sys
import time
from pathlib import Path

import numpy as np

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
sys.path.insert(0, str(ROOT / "experiments"))

from run_experiments import (  # noqa: E402
    RESULT_DIR,
    contact_sheet,
    discover_images,
    load_rgb,
    save_rgb,
)
from src.baselines import limit_long_side  # noqa: E402
from src.metrics import size_matches  # noqa: E402
from src.retarget import retarget  # noqa: E402
```

```

ORDER = [
    "brick_house",
    "butterfly",
    "child",
    "eagle",
    "Johanneskirche",
    "obama",
    "Sanfrancisco",
    "waterfall",
]
PAPER = ["Johanneskirche", "obama", "Sanfrancisco", "waterfall"]


def main() -> None:
    images = {
        path.stem: limit_long_side(load_rgb(path), 640) for path in discover_images()
    }
    check = images["Sanfrancisco"]
    height, width = check.shape[:2]
    target_75 = (height, max(2, int(round(width * 0.75))))
    ours_75, info_75 = retarget(check, target_75, method="ours", return_info=True)
    hybrid_75, info_h = retarget(check, target_75, method="hybrid", return_info=True)
    diff = int(np.abs(ours_75.astype(np.int16) - hybrid_75.astype(np.int16)).max())
    print(
        f"75% Sanfrancisco max abs diff {diff} "
        f"scaled {info_h.get('scaled_after_seams')} seams {info_75.get('vertical_seams')}",
        flush=True,
    )
    if diff != 0 or info_h.get("scaled_after_seams"):
        raise SystemExit("hybrid at 75% should match ours and should not scale")

    rows = []
    outputs = {}
    for name in ORDER:
        image = images[name]
        height, width = image.shape[:2]
        target = (height, max(2, int(round(width * 0.50))))
        outputs[(name, "original")] = image
        for method in ("scale", "ours", "hybrid"):
            print(f"{name} {method}", flush=True)
            started = time.perf_counter()
            output, info = retarget(image, target, method=method, return_info=True)
            elapsed = time.perf_counter() - started
            print(
                f"  {elapsed:.1f}s {output.shape[0]}x{output.shape[1]} "
                f"seams={info.get('vertical_seams')} scaled={info.get('scaled_after_seams')}",
                flush=True,
            )
            if not size_matches(output, target):
                raise SystemExit(f"size mismatch {name} {method}")
            outputs[(name, method)] = output
            rows.append({
                "image": name,
                "method": method,
                "target_w": target[1],
                "out_w": output.shape[1],
                "vertical_seams": info.get("vertical_seams", 0),
                "scaled_after_seams": info.get("scaled_after_seams", 0),
                "runtime_s": f"{elapsed:.4f}",
            })

    RESULT_DIR.mkdir(parents=True, exist_ok=True)
    csv_path = RESULT_DIR / "hybrid_50.csv"
    with csv_path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)

    def sheet(names: list[str]):
        records = []
        for name in names:
            records.append((name, [
                ("Original", outputs[(name, "original")]),
                ("Scale", outputs[(name, "scale")]),
                ("Ours", outputs[(name, "ours")]),
                ("Hybrid", outputs[(name, "hybrid")]),
            ]))
        return contact_sheet(records)
```

```
    save_rgb(RESULT_DIR / "fig_hybrid_50.png", sheet(PAPER))
    save_rgb(RESULT_DIR / "fig_hybrid_50_all.png", sheet(ORDER))
    print("wrote figures", flush=True)


if __name__ == "__main__":
    main()
```

Listing 11. experiments/run_hybrid.py